Gamebot | 화면을 읽고 행동하는 Android 자동화 앱

모바일게임의 보상 확인, 팝업 닫기, 가방 정리처럼 화면을 계속 지켜보며 반복하는 조작을 줄이고 싶었습니다. 정해진 좌표를 시간마다 누르면 예상하지 못한 팝업에서 잘못 동작할 수 있어, 실제 화면의 버튼과 상태를 찾은 다음 행동하도록 만들었습니다. 앱을 출시했고 이후 서비스를 종료했습니다.

AI 코딩 에이전트가 보편화되기 전인 **2020~2021년, 화면 수집·라벨링·YOLO 학습·TFLite 변환·Android 추론·상태 판단·터치 연동과 배포를 혼자 수행**했습니다. Darknet과 TensorFlow Lite 등 기존 오픈소스를 실제 게임 데이터와 모바일 환경에 맞춰 통합한 작업입니다. 탐지 알고리즘 자체를 발명했다는 의미는 아닙니다.

오프라인에서는 **게임 녹화 → 프레임 추출 → 라벨링 → Darknet 학습 → TFLite 변환**을 수행했습니다. 앱에서는 **화면 캡처 → 이미지 전처리 → 기기 내 추론 → 상태·행동 선택 → 좌표 변환 → 터치·스와이프**로 연결했습니다. 서버 추론과 AWS는 사용하지 않았습니다.

1. 실제 화면을 학습할 데이터로 만들기

작은 게임 버튼을 찾으려면 실제 플레이 환경의 이미지와 위치 정보가 필요했습니다. 영상을 녹화하고 OpenCV·Yolo_mark로 프레임을 추출한 뒤 LabelImg·Yolo_mark로 라벨을 작성했습니다. 고정 위치 UI는 Python 스크립트로 반복 라벨 작성을 보조하고, 게임·화면 종류별로 자료를 정리했습니다.

초기 SSD MobileNet 실험 이후 YOLOv4-tiny 계열을 사용했습니다. Darknet에서 학습한 가중치를 SavedModel을 거쳐 TFLite로 변환하고 앱에 모델과 라벨을 함께 넣었습니다. 데이터 준비부터 Android 실행까지 직접 연결한 것이 이 작업의 중심입니다.

당시 검증 목록은 화면 폴더 안에서 무작위로 나눴으므로 같은 녹화의 유사 프레임이 양쪽에 섞일 수 있습니다. 독립된 녹화·기기·게임 버전으로 평가한 재현 자료가 충분하지 않아 정확도 수치를 성과로 제시하지 않습니다.

2. 추론 속도보다 빨리 쌓이는 화면을 다루기

게임은 계속 진행되는데 과거 프레임이 쌓이면 이미 사라진 팝업을 뒤늦게 누를 수 있습니다. 모든 프레임 보존보다 현재 화면에 맞는 행동이 중요해 ImageReader 버퍼를 한 장으로 제한하고 추론을 직렬 실행했습니다.

초기에는 화면을 JPEG 파일로 저장한 뒤 다시 읽었습니다. 이 중간 저장이 필요하지 않아 Bitmap을 추론기에 직접 전달하도록 바꿔 인코딩·파일 쓰기·재읽기를 제거했습니다. 불필요한 I/O를 없앤 변경은 코드로 확인되지만, 전후 속도의 정량 비교는 남아 있지 않습니다.

작은 입력은 빨라도 작은 버튼을 놓쳤습니다. 게임별 입력 크기를 시험하고 대표 SK2 설정에는 448×448을 사용했습니다. FP16 모델과 CPU thread 수도 조정했으나, 이를 현재 기기의 보장 FPS나 개선률로 표현하지 않습니다.

3. 버튼을 찾는 것과 눌러야 할 때를 구분하기

홈 버튼은 가방 화면에도 분해 결과 화면에도 나타납니다. 탐지 점수가 가장 높은 객체를 누르는 방식으로는 다음 행동을 결정할 수 없었습니다. 한 화면에서 찾은 객체들의 조합을 규칙과 비교하고, 직전 행동에 따라 다음에 우선할 대상을 정했습니다.

SK2에서는 인벤토리 가득 참 탐지부터 장비 선택·분해·확인·결과 처리 후 절전 화면 복귀까지 연결했습니다. 화면 전환 중 잠깐 나타난 객체에 바로 반응하지 않도록 최근 탐지의 반복 일치를 확인하고, 실제 추론시간에 맞춰 이력을 참조하는 범위를 조정했습니다. 이미 누른 버튼이 남아 있어도 연속 클릭하지 않도록 행동 이력을 사용했습니다.

객체 탐지는 모델이, 행동 선택은 직접 작성한 규칙이 담당했습니다. 게임 업데이트로 화면이 바뀌면 모델과 규칙을 함께 손봐야 했고, 모든 예외 화면을 자동 처리하지는 못했습니다.

4. 모델의 작은 화면 좌표를 실제 터치로 옮기기

모델 입력의 좌표를 실제 화면에 그대로 적용하면 다른 곳을 누릅니다. 축소·자르기에 사용한 변환을 반대로 적용해 탐지 상자의 중심을 원래 화면 좌표로 옮기고 AccessibilityService로 터치와 스와이프를 실행했습니다.

회전 후에는 기존 가로·세로 크기가 맞지 않아 VirtualDisplay·ImageReader와 좌표 변환을 다시 구성했습니다. 이동 가능한 제어창과 광고 overlay도 화면의 일부였으므로 현재 영역을 계산해 그 안으로 향하는 터치를 막았습니다. 좌표 계산만 고치는 대신 캡처부터 입력까지 같은 기준을 유지하는 변경이었습니다.

5. 여러 게임에 공통 엔진을 쓰고 종료 자원을 정리하기

화면 캡처·추론·overlay·접근성 입력은 공용 Android 런타임에 두고, 게임별 앱은 모델·입력 크기·라벨과 동작 규칙을 연결하도록 나눴습니다. 바람의나라·왕좌의게임·스톤에이지·SK2 등에서 게임마다 앱 전체를 다시 만드는 비용을 줄이려는 구조였습니다. 게임별 완성 기능은 서로 달랐습니다.

앱을 닫아도 캡처나 추론이 남을 수 있어 종료 시 추론 thread, TFLite, ImageReader, VirtualDisplay, MediaProjection, receiver와 overlay를 명시적으로 해제했습니다. 배포는 로컬 서명 빌드와 Google Play 수동 업로드였습니다. 자동 CI/CD나 장기 누수 검증까지 구축한 것은 아닙니다.

현재 서비스는 종료됐으며, 공개본에는 원본 이미지·모델·공용 모듈이 완전하게 들어 있지 않습니다. 현재 Android에서 그대로 빌드·재학습되는 상태라고 소개하지 않습니다. 남은 코드는 당시 구현과 문제 해결 경로를 확인하는 자료입니다.

확인 자료

대표 앱과 기술 사례 · 공용 Android 런타임 · 학습·변환 기록

바람의나라 실행 영상 · 라오킹 실행 영상 · 왕좌의게임 실행 영상

영상은 같은 Gamebot 계열의 실행 사례입니다. SK2 전용 실행 영상으로는 제시하지 않습니다.